

Error Pattern Analysis in an Online Judge System

Reproducing and Extending Shimizu et al. (2025)

Raphael BELY

August 28, 2026

DDSE Laboratory — Ritsumeikan University

About This Internship

- Research internship — DDSE Laboratory, Ritsumeikan University
- Supervised by Prof. Makihara Erina
- Goal: reproducing, then extending
Shimizu, Makihara & Yoshida (2025) — An Empirical Study of the Error Characteristics in an Online Judge System, FSE Companion '25.
- May 28 – September 20, 2026 (16 weeks)
- French student, Enseirb-Matmeca (Bordeaux INP) — 4-year engineering program, 1 year left after this, AI specialization



The Dataset

- IBM Research (2021) — built by scraping two online judges: **AIZU** and **AtCoder**
- Official CodeNet total (both sources combined): 4,053 problems, 13,916,868 submissions

Puri et al. 2021, arXiv:2105.12655

- **This project analyzes:** the AtCoder side only, restricted to ABC contests — 659 problems (difficulty A–F), ~8.9M submissions, 96,207 users
- One file = one submission, path encodes everything:
data/(problem_id)/(language)/(submission_id).ext
e.g. data/p00023/C++/s006384060.cpp

What We Measure

Submission verdict

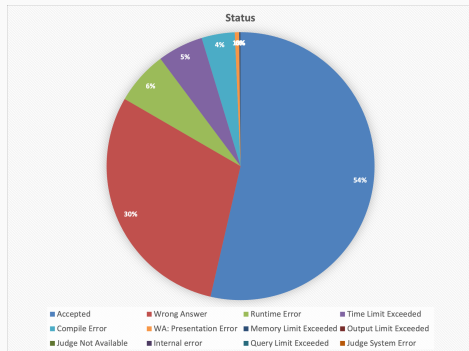
- AC — Accepted
- WA — Wrong Answer
- TLE — Time Limit Exceeded
- RE — Runtime Error
- CE — Compile Error

Also tracked: problem difficulty, A (easy)
→ F (hard)

**One real row, same order as the file path
above:**

problem p02659 (C) → Python →
s718190921 → **WA** + **code**

Raphael BELY — DDSE Lab, Ritsumeikan



Official CodeNet-wide verdict breakdown (Puri et al. 2021, arXiv:2105.12655) — all languages/sources pooled, not just our AtCoder/Python subset.

Research Questions

RQ1 — Error Patterns

How do programmers' error patterns vary with problem difficulty and user proficiency?

RQ2 — Code-Level Semantic Analysis

Do codes that are close together in vector space share the same error pattern?

RQ3 — Prediction (optional)

Can RQ1/RQ2 insights function as a predictive model? — still open, this is what the post-Aug-28 LLM exploration will test

Methodology

General Approach

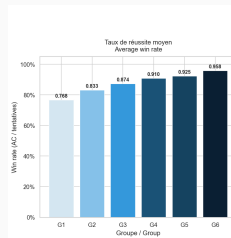
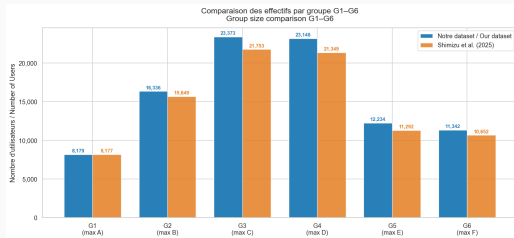
- **RQ1:** classify users (G1–G6), then cross-tabulate error rates by difficulty
- **RQ2:** embed code, test whether neighbors in that space share the same verdict
- **The rule behind every number in this talk:** never report one in isolation — always against a reference (a random baseline, chance, or Shimizu's own numbers)

Results — RQ1

How do programmers' error patterns vary with problem difficulty and user proficiency?

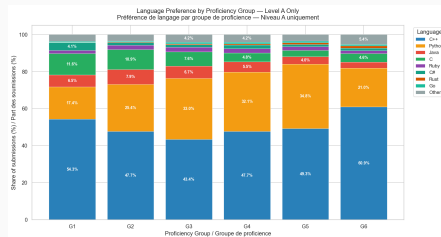
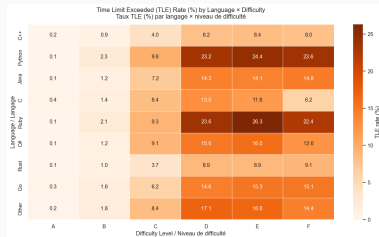
Preliminary — G1–G6 Classification

- **Question:** can users be sorted into meaningful proficiency groups?
- Not a CodeNet field — this is **step 1 of Shimizu's method**, reproduced here first
- **Method:** group = hardest difficulty ever solved
- **Result:** monotonic trends G1→G6 — win rate 76.8%→95.8% — validated against Shimizu Table 3 (+6–8% gap, explained)



Quick Detour: Language Matters — Not as Expected

- **Question:** does language choice influence the error type?
- **Result 1:** TLE $\sim 3\times$ more frequent in Python than C++ (23.2% vs 8.2%, level D)
- **Result 2 (counter-intuitive):** beginners (G1) do **not** mostly use Python — 54.3% in C++

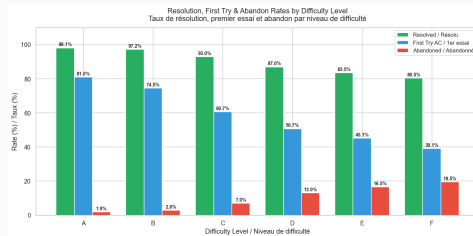
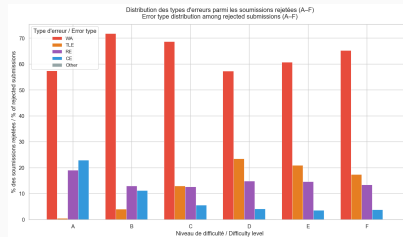


CE $\leftrightarrow$ TLE inversion

- AC rate: 73% $\rightarrow$ 35% (A $\rightarrow$ F)
- Replicates Shimizu Figure 4

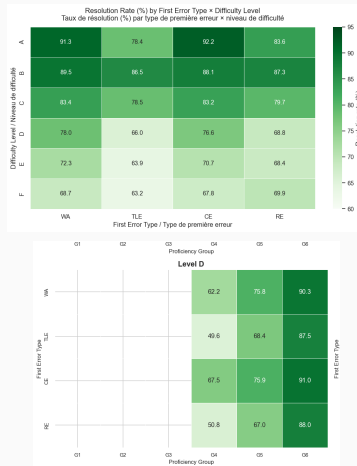
Resolution & first try

- Resolution: 98.1% $\rightarrow$ 80.5% (A $\rightarrow$ F)
- First-try AC falls faster:
81.0% $\rightarrow$ 39.1% (A $\rightarrow$ F)



Expertise Flattens the Effect of Error Type

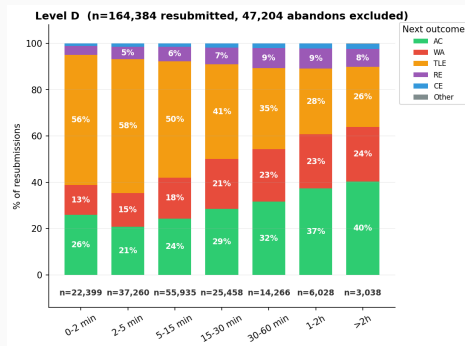
- **Question:** does the type of a user's first error predict final success, the same way for everyone?
- TLE = worst first-error signal, at **every** level without exception (pooled across groups)
- **But** that gap depends on skill:
best/worst error-type resolution gap —
18pp for G4, only 3.5pp for G6 (level D)



Bottom: level D only (full grid: NB06, Sec. 4)

The *Blind Patch* Anomaly

- **Question:** does the delay before resubmitting after a TLE change the odds of success?
- **Result:** resubmitting within 2–5 min is **worse** than either faster or later ($n > 37,000$, level D)
- The only result that wasn't predictable *a priori* (stated as such in the notebook itself)



Level D panel shown (NB08, S1, full grid covers all 6 levels).

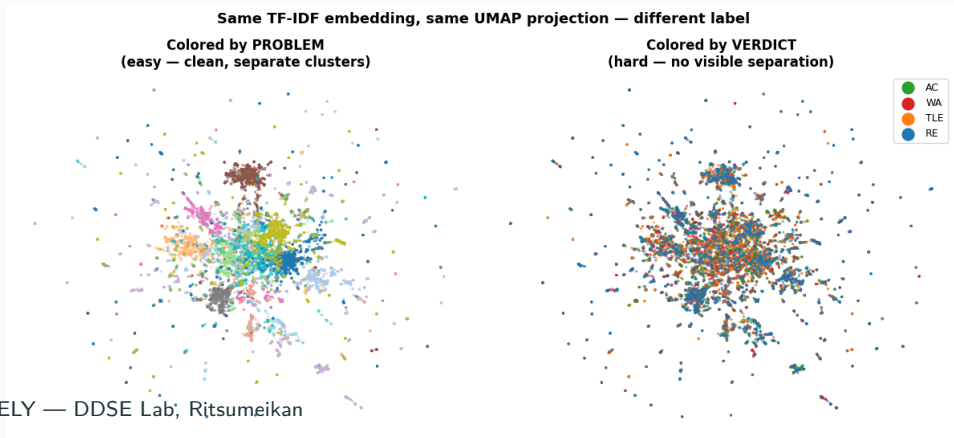
- Error patterns depend on language, expertise, and even resubmission behavior
- Throughline: expertise doesn't just lower the error rate — it lowers **how much the error type matters**

Results — RQ2

Do codes that are close together in vector space share the same error pattern?

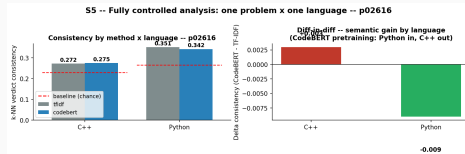
Sanity Check: Is the Embedding Credible At All?

- Before asking whether it captures *verdict*: does the embedding capture *anything* recognizable?
- Yes — problem identity is trivial to see; verdict is not, on the exact same points



CodeBERT vs TF-IDF

- **Question:** does a semantic embedding capture the code $\leftrightarrow$ verdict link better than plain TF-IDF?
- **Method:** k-NN lift (k=10, cosine), control funnel language $\rightarrow$ problem $\rightarrow$ both
- **Result:** once fully controlled, CodeBERT provides no advantage — even in Python ($\times 1.33$ TF-IDF vs $\times 1.29$ CodeBERT)

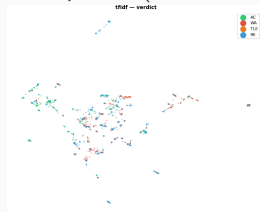


S5, single-problem/single-language control (p02616). Right panel: CodeBERT — TF-IDF, Python is negative.

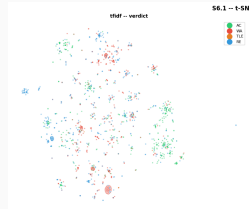
Generalization — and a Methodological Twist

- Confirmed across 12 problems (21,773 submissions) — on **verdict** (the actual RQ2 question), TF-IDF and CodeBERT are near tied ($\times 1.37$ vs $\times 1.35$, 7/12 vs 5/12 wins); group is where TF-IDF pulls ahead, 11/12
- **Twist:** UMAP shows **no** visible separation — but t-SNE + a confusion heatmap reveal a real, highly localized signal (AC/WA 69%/68% neighbor consistency, TLE nearly invisible at 3%)

UMAP — p02659 (TF-IDF, verdict)

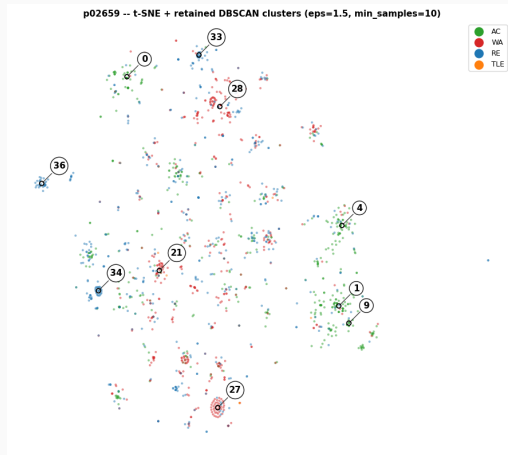


t-SNE — same data



Zooming In — Does Proximity Reflect Real Similarity?

- **Question raised by the professors, Aug. 7:** do these clusters in the t-SNE projection reflect genuinely similar code, or an embedding coincidence?
- **Method:** automatic cluster detection (DBSCAN, not by eye) → real code similarity (difflib) compared to a random baseline of matching verdict composition



10 automatically-detected clusters, p02659 (NB14, S1). Numbers = cluster IDs used in the S3/S6 tables.

Result — the Exception Wasn't an Embedding Failure

- **9/10 clusters**: code genuinely more similar than the baseline (up to $2.73\times$)
- **Cluster 36** ($0.83\times$, RE, 100% pure — the one spotted *by eye* first): checked all 26 files, not a sample
- **26/26 are C++**, every single one labeled “Python” in the dataset — not the embedding failing on comparable code, a **data-labeling bug** (NB14, S7)

Cluster 34 ($2.73\times$) — s144425654 / s828321135:

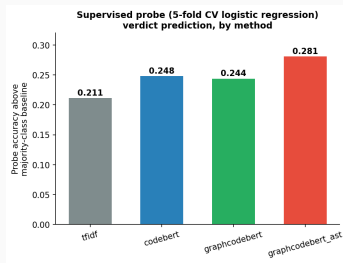
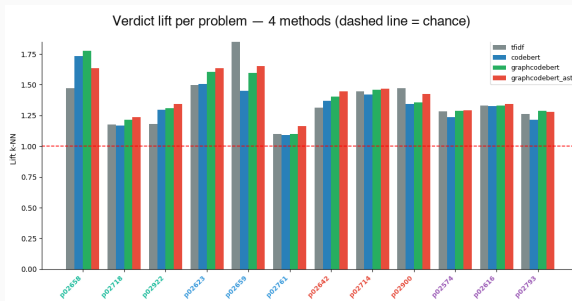
```
a, b = map(int,input().split())
print(int(a*b))
a,b=map(int,input().split())
print(int(a*b))
```

s392220690, cluster 36 ($0.83\times$):

```
a, b = input()
print("%lld", a * b)#include <iostream>
#include <cstdio>
...
int main() {
    ...
}
```

Pushing the Model Comparison Further

- **Question:** does a model aware of code structure (GraphCodeBERT, real data-flow graph) do better?
- **Result:** mixed — no version clearly beats TF-IDF on k-NN (top), but a supervised probe puts the real-graph version ahead on verdict (bottom)
- No clear winner — an open thread for the next steps



- A real but modest verdict signal, confirmed by 3 independent methods (k-NN lift, t-SNE+heatmap, real code via NB14)
- No more sophisticated model tested clearly beat TF-IDF for this specific task

Analysis and Outlook

Summary and Next Steps

- RQ1: rich error patterns, shaped by language, expertise, and resubmission behavior
- RQ2: a real but modest signal — validated at the level of actual code, not just the embedding
- **Next (3 weeks after August 28):** LLM exploration — a natural direction after finding that neither CodeBERT nor GraphCodeBERT clearly beat TF-IDF
- Also on the roadmap, not started yet: RQ3 (error prediction) and an interactive dashboard

Thank you!

Questions?



`github.com/Raphael-Bely/OJS-submission-analytics`